

Python Code

```

1  #!/usr/bin/env python3
2  """
3  linkedin_people_scraper.py
4  Collects employees listed on a LinkedIn company "People" page and APPENDS them
5  to a CSV (and optionally a SQLite table) with the columns:
6      company_name | first_name | last_name | job_title | linkedin_profile_url | email
7  - job_title: most recent title held at the queried company, taken from the
8    profile's Experience section. Company names are matched fuzzily (suffixes like
9    LLC / Inc / Corp / Corporate / Company / Co / Ltd are ignored), so
10     "weverse-corporate" matches "WEVERSE COMPANY".
11  - email: only filled if visible in the profile's "Contact info" overlay for the
12    logged-in account.
13  Usage:
14      python linkedin_people_scraper.py "https://www.linkedin.com/company/acme/people/" \
15        --out all_people.csv --sqlite all_people.db --max-people 50
16      # Check how two company names compare (no network access):
17      python linkedin_people_scraper.py --test-match weverse-corporate "WEVERSE COMPANY"
18  On first run, a browser window opens; log in to LinkedIn manually, then press
19  Enter in the terminal. The session is saved in .li_browser_profile for reuse.
20  NOTE: Automated scraping violates LinkedIn's User Agreement and may get the
21  account restricted or banned. Scraped personal data is subject to privacy laws
22  (GDPR, CCPA, etc.). Use responsibly and at your own risk.
23  """
24  import argparse
25  import csv
26  import random
27  import re
28  import sqlite3
29  import sys
30  import time
31  import unicodedata
32  from difflib import SequenceMatcher
33  from pathlib import Path
34  from urllib.parse import urlparse
35  from playwright.sync_api import sync_playwright, TimeoutError as PWTimeout
36  COLUMNS = ["company_name", "first_name", "last_name", "job_title",
37             "linkedin_profile_url", "email"]
38  EMAIL_RE = re.compile(r"[A-Za-z0-9_%+~\.\-]\+@[A-Za-z0-9\.\-]+\.[A-Za-z]{2,}")
39  # ----- #
40  # General helpers
41  # ----- #
42  def polite_sleep(lo: float, hi: float) -> None:
43      time.sleep(random.uniform(lo, hi))
44  def normalize_people_url(url: str) -> tuple[str, str]:
45      """Return (people_url, company_slug)."""
46      if not url.startswith("http"):
47          url = "https://" + url
48      parsed = urlparse(url)
49      m = re.search(r"/company/([^\/]*)", parsed.path)
50      if not m:
51          sys.exit(f"Could not find '/company/<name>' in URL: {url}")
52      slug = m.group(1)
53      return f"https://www.linkedin.com/company/{slug}/people/", slug
54  def clean_profile_url(href: str) -> str | None:
55      """Normalize to https://www.linkedin.com/in/<slug>"""
56      m = re.search(r"linkedin\.com/in/([^\/?#]*)", href)
57      if not m:
58          return None
59      return f"https://www.linkedin.com/in/{m.group(1)}/"
60  def split_name(full_name: str) -> tuple[str, str]:
61      """Strip credentials/emoji/parentheticals and split into first/last."""
62      name = full_name.strip()
63      name = re.sub(r"(\.?\s)", "", name) # "Jane (JL) Doe" -> "Jane Doe"
64      name = name.split("(")[0] # "Jane Doe, MBA" -> "Jane Doe"
65      name = re.sub(r"[^\w\s\.-]", "", name) # drop emoji / symbols
66      name = re.sub(r"[\s+]", " ", name).strip()
67      parts = name.split(" ")
68      if not parts or not parts[0]:
69          return "", ""
70      if len(parts) == 1:
71          return parts[0], ""
72      return parts[0], " ".join(parts[1:])
73  # ----- #
74  # Company-name matching
75  # ----- #
76  # Trailing tokens stripped from company names (repeatedly, from the end only).
77  # Multi-word entries are matched as whole phrases.
78  COMPANY_SUFFIXES = [
79      # multi-word legal forms
80      "public limited company", "limited liability company", "limited liability co",
81      "private limited", "co ltd", "pvt ltd", "pte ltd", "pty ltd", "sdn bhd",
82      "co inc", "and co", "and company",
83      # single-word legal forms
84      "incorporated", "corporation", "corporate", "company", "limited",
85      "llc", "llp", "lp", "inc", "corp", "co", "ltd", "plc", "pc",
86      "gmbh", "ag", "kg", "sa", "sas", "sarl", "srl", "spa", "bv", "nv",
87      "ab", "as", "asa", "oy", "kk", "pte", "pvt", "pty", "bhd",
88      # common descriptors
89      "holdings", "holding", "group", "international", "intl", "global",
90  ]
91  _SUFFIX_TOKENS = sorted((tuple(s.split()) for s in COMPANY_SUFFIXES),
92                          key=len, reverse=True)
93  def normalize_company(name: str) -> str:
94      """

```

```

95     Reduce a company name to its 'core' for comparison, e.g.
96     'WEVERSE COMPANY'      -> 'weverse'
97     'weverse-corporate'    -> 'weverse'
98     'Weverse Co., Ltd.'    -> 'weverse'
99     'The Walt Disney Company' -> 'walt disney'
100     Unicode letters (e.g. Korean, Japanese) are preserved.
101     """
102     s = unicodedata.normalize("NFKD", name or "")
103     s = "".join(c for c in s if not unicodedata.combining(c)).lower()
104     s = re.sub(r"(\.|\?)\.[^\s]*", " ", s) # drop parentheticals
105     s = s.replace("&", " and ")
106     s = re.sub(r"[\W_]+", " ", s) # punctuation/hyphens -> space
107     tokens = s.split()
108     if len(tokens) > 1 and tokens[0] == "the":
109         tokens = tokens[1:]
110     changed = True
111     while changed and len(tokens) > 1:
112         changed = False
113         for suf in _SUFFIX_TOKENS:
114             n = len(suf)
115             if len(tokens) > n and tuple(tokens[-n:]) == suf:
116                 tokens = tokens[:-n]
117                 changed = True
118             break
119     return " ".join(tokens)
120 def slug_to_name(slug: str) -> str:
121     """'weverse-corporate' -> 'weverse corporate'; 'acme-inc-2' -> 'acme inc'"""
122     parts = [p for p in re.split(r"[-_]+", slug.lower()) if p]
123     if len(parts) > 1 and parts[-1].isdigit():
124         parts = parts[:-1]
125     return " ".join(parts)
126 class CompanyMatcher:
127     """Decides whether an experience entry refers to the target company."""
128     def __init__(self, names: list[str], company_id: str | None, slug: str | None,
129                 threshold: float = 0.88, min_fuzzy_len: int = 5,
130                 allow_prefix: bool = False):
131         self.company_id = company_id
132         self.slug = (slug or "").lower()
133         self.threshold = threshold
134         self.min_fuzzy_len = min_fuzzy_len
135         self.allow_prefix = allow_prefix
136         cores = {normalize_company(n) for n in names if n}
137         cores.discard("")
138         # list of (core, compact) e.g. ('walt disney', 'waltdisney')
139         self.targets = [(c, c.replace(" ", "")) for c in sorted(cores)]
140     def describe(self) -> str:
141         return " ".join(core for core, _ in self.targets) or "(none)"
142     def score_name(self, name: str) -> float:
143         """Similarity in [0, 1] between 'name' and the best target core."""
144         core = normalize_company(name)
145         if not core:
146             return 0.0
147         compact = core.replace(" ", "")
148         best = 0.0
149         for t_core, t_compact in self.targets:
150             # Exact match after normalization (spaces ignored: 'we verse' == 'weverse')
151             if compact == t_compact:
152                 return 1.0
153             # Too short for fuzzy matching -> exact only
154             if min(len(compact), len(t_compact)) < self.min_fuzzy_len:
155                 continue
156             best = max(best, SequenceMatcher(None, compact, t_compact).ratio())
157             # Optional: 'samsung' matches 'samsung electronics' (or vice versa)
158             if self.allow_prefix:
159                 a, b = core.split(), t_core.split()
160                 shorter, longer = (a, b) if len(a) <= len(b) else (b, a)
161                 if longer[:len(shorter)] == shorter:
162                     best = max(best, 0.90)
163         return best
164     def match_name(self, name: str) -> str | None:
165         """Return a human-readable match reason, or None."""
166         score = self.score_name(name)
167         if score >= self.threshold:
168             return f"name '{name}' (score {score:.2f})"
169         return None
170     def match_link(self, href: str) -> str | None:
171         """Match on a /company/<id-or-slug>/ link. Returns reason or None."""
172         m = re.search(r"/company/([^\s#]+)", href)
173         if not m:
174             return None
175         seg = m.group(1).lower()
176         if self.company_id and seg == self.company_id:
177             return f"company id {seg}"
178         if self.slug and seg == self.slug:
179             return f"company slug {seg}"
180         if not seg.isdigit():
181             score = self.score_name(slug_to_name(seg))
182             if score >= self.threshold:
183                 return f"company slug '{seg}' (score {score:.2f})"
184         return None
185     # ----- #
186     # Storage
187     # ----- #
188     def migrate_csv(csv_path: Path) -> None:
189         """If an existing CSV has an older header, rewrite it with the current COLUMNS."""
190         if not csv_path.exists() or csv_path.stat().st_size == 0:
191             return
192         with csv_path.open(newline="", encoding="utf-8") as f:

```

```

193     reader = csv.DictReader(f)
194     if reader.fieldnames == COLUMNS:
195         return
196     rows = list(reader)
197     print(f"Migrating {csv_path} to new column layout: {COLUMNS}")
198     tmp = csv_path.with_suffix(csv_path.suffix + ".tmp")
199     with tmp.open("w", newline="", encoding="utf-8") as f:
200         writer = csv.DictWriter(f, fieldnames=COLUMNS, restval="", extrasaction="ignore")
201         writer.writeheader()
202         writer.writerows(rows)
203     tmp.replace(csv_path)
204 def load_existing(csv_path: Path) -> dict[tuple[str, str], dict]:
205     """Return {(company_name, profile_url): row} for rows already in the CSV."""
206     if not csv_path.exists():
207         return {}
208     with csv_path.open(newline="", encoding="utf-8") as f:
209         return {
210             (row["company_name"], row["linkedin_profile_url"]): row
211             for row in csv.DictReader(f)
212         }
213 def append_csv(csv_path: Path, row: dict) -> None:
214     """Append one row, writing the header only if the file is new/empty."""
215     is_new = not csv_path.exists() or csv_path.stat().st_size == 0
216     with csv_path.open("a", newline="", encoding="utf-8") as f:
217         writer = csv.DictWriter(f, fieldnames=COLUMNS)
218         if is_new:
219             writer.writeheader()
220             writer.writerow(row)
221 def init_sqlite(db_path: Path) -> None:
222     """Create the table if needed and add any missing columns (e.g. job_title)."""
223     con = sqlite3.connect(db_path)
224     con.execute(
225         """CREATE TABLE IF NOT EXISTS employees (
226             company_name TEXT NOT NULL,
227             first_name TEXT,
228             last_name TEXT,
229             job_title TEXT,
230             linkedin_profile_url TEXT NOT NULL,
231             email TEXT,
232             PRIMARY KEY (company_name, linkedin_profile_url)
233         )"""
234     )
235     existing_cols = {r[1] for r in con.execute("PRAGMA table_info(employees)")}
236     if "job_title" not in existing_cols:
237         print(f"Adding job_title column to {db_path}")
238         con.execute("ALTER TABLE employees ADD COLUMN job_title TEXT")
239     con.commit()
240     con.close()
241 def write_sqlite(db_path: Path, rows: list[dict]) -> None:
242     con = sqlite3.connect(db_path)
243     con.executemany(
244         """INSERT INTO employees
245             (company_name, first_name, last_name, job_title, linkedin_profile_url, email)
246             VALUES (:company_name, :first_name, :last_name, :job_title,
247                 :linkedin_profile_url, :email)
248             ON CONFLICT(company_name, linkedin_profile_url) DO UPDATE SET
249                 first_name = excluded.first_name,
250                 last_name = excluded.last_name,
251                 job_title = COALESCE(excluded.job_title, employees.job_title),
252                 email = COALESCE(excluded.email, employees.email)"""
253         , rows,
254     )
255     con.commit()
256     con.close()
257 # ----- #
258 # Browser steps
259 # ----- #
260 def ensure_logged_in(page) -> None:
261     page.goto("https://www.linkedin.com/feed/", wait_until="domcontentloaded")
262     polite_sleep(2, 3)
263     if any(x in page.url for x in ("login", "authwall", "checkpoint", "signup")):
264         print("\n>> Please log in to LinkedIn in the opened browser window.")
265         input(">>> Press Enter here once you see your LinkedIn feed... ")
266     page.goto("https://www.linkedin.com/feed/", wait_until="domcontentloaded")
267     if "feed" not in page.url:
268         sys.exit("Still not logged in; aborting.")
269 def get_company_name(page, fallback: str) -> str:
270     try:
271         h1 = page.locator("h1").first
272         h1.wait_for(timeout=8000)
273         text = h1.inner_text().strip()
274         return text or fallback
275     except PWTimeout:
276         return fallback
277 def get_company_id(page) -> str | None:
278     """
279     Best-effort extraction of the company's numeric ID from the company page.
280     Experience entries on profiles usually link to /company/<numeric-id>/.
281     """
282     html = page.content()
283     for pattern in (r"urn:li:fsd_company:(\d+)",
284                    r"urn:li:company:(\d+)",
285                    r"f_C=(\d+)",
286                    r'"companyId"\s*:\s*(\d+)'):
287         m = re.search(pattern, html)
288         if m:
289             return m.group(1)
290     return None

```

```

291 # JS run on the People page: find each person card and pull profile URL + name.
292 EXTRACT_PEOPLE_JS = """
293 () => {
294   const out = [];
295   const anchors = document.querySelectorAll('main a[href*="/in/"]');
296   for (const a of anchors) {
297     const card = a.closest('li') || a.parentElement;
298     let name = '';
299     const titleEl = card && (
300       card.querySelector('.artdeco-entity-lockup__title') ||
301       card.querySelector('.org-people-profile-card__profile-title')
302     );
303     if (titleEl) name = titleEl.innerText;
304     if (!name) name = (a.innerText || a.getAttribute('aria-label') || '');
305     name = name.split('\n').map(s => s.trim()).filter(Boolean)[0] || '';
306     out.push({ href: a.href, name });
307   }
308   return out;
309 }
310 """
311 # JS run on /in/<slug>/details/experience/: return raw top-level experience
312 # entries. Matching against the target company happens in Python.
313 EXTRACT_EXPERIENCE_JS = """
314 () => {
315   // Visible text lines. LinkedIn duplicates text in visually-hidden spans for
316   // screen readers; aria-hidden spans are the visible copies.
317   const lines = el => {
318     const spans = Array.from(el.querySelectorAll('span[aria-hidden="true"]'))
319       .map(s => s.innerText.trim()).filter(Boolean);
320     if (spans.length) return spans;
321     return (el.innerText || '').split('\n').map(s => s.trim()).filter(Boolean);
322   };
323   // Date-range line, e.g. "Jan 2021 - Present · 3 yrs 2 mos"
324   const isDateLine = s =>
325     /\b(19[20]\d{2}\b|present)/i.test(s) && /\s[-\u2013]\s/.test(s);
326   const topItems = Array.from(document.querySelectorAll('main section li'))
327     .filter(li => li.parentElement.closest('main section li'));
328   return topItems.map(li => {
329     // Nested roles (grouped layout): direct child <li>s containing a date range
330     const nested = Array.from(li.querySelectorAll('li')).filter(sub =>
331       sub.parentElement.closest('li') === li &&
332       lines(sub).slice(0, 4).some(isDateLine)
333     );
334     return {
335       lines: lines(li).slice(0, 6),
336       hrefs: Array.from(li.querySelectorAll('a[href*="/company/"]')).map(a => a.href),
337       nested_titles: nested.map(sub => lines(sub)[0]).filter(Boolean),
338     };
339   }).filter(e => e.lines.length);
340 }
341 """
342 def collect_people(page, people_url: str, max_new: int, known_urls: set[str]) -> dict[str, str]:
343   """Scroll the People page until 'max_new' previously-unseen people are found."""
344   page.goto(people_url, wait_until="domcontentloaded")
345   polite_sleep(3, 5)
346   people: dict[str, str] = {}
347   stale_rounds = 0
348   while stale_rounds < 4:
349     before = len(people)
350     for item in page.evaluate(EXTRACT_PEOPLE_JS):
351       url = clean_profile_url(item["href"])
352       name = item["name"].strip()
353       # Out-of-network members show as "LinkedIn Member" with no usable URL
354       if url and name and name.lower() != "linkedin member":
355         people.setdefault(url, name)
356     new_count = sum(1 for u in people if u not in known_urls)
357     print(f" ...{len(people)} seen, {new_count} new")
358     if new_count >= max_new:
359       break
360     show_more = page.locator("button:has-text('Show more results')")
361     if show_more.count() and show_more.first.is_visible():
362       show_more.first.click()
363     else:
364       page.mouse.wheel(0, 3000)
365     polite_sleep(2.5, 4.5)
366     stale_rounds = stale_rounds + 1 if len(people) == before else 0
367     new_only = {u: n for u, n in people.items() if u not in known_urls}
368     return dict(list(new_only.items()))[:max_new]
369 def pick_title(entries: list[dict], matcher: CompanyMatcher) -> tuple[str | None, str | None]:
370   """
371   Walk experience entries (newest first) and return (title, match_reason)
372   for the first entry that refers to the target company.
373   """
374   for e in entries:
375     lines = e.get("lines") or []
376     nested = e.get("nested_titles") or []
377     grouped = bool(nested)
378     # Where the company name lives depends on the layout:
379     #   grouped: line 0 = company, roles are nested
380     #   single: line 0 = title, line 1 = "Company · Full-time"
381     company_lines = lines[:1] if grouped else lines[1:3]
382     reason = None
383     for href in e.get("hrefs") or []:
384       reason = matcher.match_link(href)
385       if reason:
386         break
387     if not reason:
388       for line in company_lines:

```

```

389         company_text = line.split("\u00b7")[0].strip() # drop ". Full-time"
390         reason = matcher.match_name(company_text)
391         if reason:
392             break
393         if not reason:
394             continue
395         title = nested[0] if grouped else (lines[0] if lines else None)
396         if title:
397             return re.sub(r"\s+", " ", title).strip(), reason
398     return None, None
399 def get_job_title(page, profile_url: str, matcher: CompanyMatcher) -> str | None:
400     """Return the most recent job title at the target company from the Experience page."""
401     try:
402         page.goto(profile_url + "details/experience/", wait_until="domcontentloaded")
403         page.locator("main section li").first.wait_for(timeout=8000)
404     except PWTimeout:
405         return None
406     page.mouse.wheel(0, 1500) # nudge lazy-loaded content
407     polite_sleep(1.0, 2.0)
408     try:
409         entries = page.evaluate(EXTRACT_EXPERIENCE_JS)
410     except Exception as e:
411         print(f" ! error reading experience: {e}")
412         return None
413     title, reason = pick_title(entries, matcher)
414     if title:
415         print(f" matched via {reason}")
416     return title
417 def get_email(page, profile_url: str) -> str | None:
418     """Open the Contact info overlay and return a visible email, if any."""
419     try:
420         page.goto(profile_url + "overlay/contact-info/", wait_until="domcontentloaded")
421         mailto = page.locator("a[href='mailto:']").first
422         mailto.wait_for(timeout=6000)
423         href = mailto.get_attribute("href") or ""
424         email = href.replace("mailto:", "").split("?")[0].strip()
425         if EMAIL_RE.fullmatch(email):
426             return email
427     except PWTimeout:
428         pass
429     except Exception as e:
430         print(f" ! error reading contact info: {e}")
431     # Fallback: scan the dialog text
432     try:
433         dialog = page.locator("[role='dialog']").first
434         if dialog.count():
435             m = EMAIL_RE.search(dialog.inner_text())
436             if m:
437                 return m.group(0)
438     except Exception:
439         pass
440     return None
441 # ----- #
442 # Main
443 # ----- #
444 def main() -> None:
445     ap = argparse.ArgumentParser(description="Scrape a LinkedIn company People page.")
446     ap.add_argument("people_url", nargs="?",
447                     help="e.g. https://www.linkedin.com/company/acme/people/")
448     ap.add_argument("--out", default="people_data.csv", help="CSV output path (appended to)")
449     ap.add_argument("--sqlite", help="Optional SQLite DB path (table: employees.db)")
450     ap.add_argument("--max-people", type=int, default=100,
451                     help="Max NEW profiles to process per run (default 100)")
452     ap.add_argument("--skip-emails", action="store_true",
453                     help="Don't open each profile's Contact info overlay")
454     ap.add_argument("--skip-titles", action="store_true",
455                     help="Don't open each profile's Experience section")
456     ap.add_argument("--alias", action="append", default=[],
457                     help="Extra company name to match in Experience (repeatable), "
458                          "e.g. --alias 'beNX' for a former name")
459     ap.add_argument("--match-threshold", type=float, default=0.88,
460                     help="Fuzzy similarity threshold 0-1 for company names (default 0.88)")
461     ap.add_argument("--allow-prefix-match", action="store_true",
462                     help="Also match when one name is a word-prefix of the other "
463                          "(e.g. 'Samsung' ~ 'Samsung Electronics'). Riskier.")
464     ap.add_argument("--test-match", nargs=2, metavar=("TARGET", "CANDIDATE"),
465                     help="Print normalization/score for two company names and exit")
466     ap.add_argument("--min-delay", type=float, default=4.0)
467     ap.add_argument("--max-delay", type=float, default=9.0)
468     ap.add_argument("--profile-dir", default="li_browser_profile",
469                     help="Where the persistent browser session is stored")
470     args = ap.parse_args()
471     # --- Offline name-matching test ---
472     if args.test_match:
473         target, candidate = args.test_match
474         m = CompanyMatcher([target, slug_to_name(target)], None, None,
475                             threshold=args.match_threshold,
476                             allow_prefix=args.allow_prefix_match)
477         print(f"target -> {m.describe()}")
478         print(f"candidate -> {normalize_company(candidate)!r}")
479         print(f"score = {m.score_name(candidate):.2f} ")
480         print(f"({'MATCH' if m.match_name(candidate) else 'no match'}) "
481               f"at threshold {args.match_threshold}")
482         return
483     if not args.people_url:
484         ap.error("people_url is required (unless using --test-match)")
485     people_url, slug = normalize_people_url(args.people_url)
486     csv_path = Path(args.out)

```

```

487 db_path = Path(args.sqlite) if args.sqlite else None
488 migrate_csv(csv_path)
489 if db_path:
490     init_sqlite(db_path)
491 existing = load_existing(csv_path) # {(company, url): row}
492 with sync_playwright() as p:
493     ctx = p.chromium.launch_persistent_context(
494         user_data_dir=args.profile_dir,
495         headless=False,
496         viewport={"width": 1280, "height": 900},
497     )
498     page = ctx.pages[0] if ctx.pages else ctx.new_page()
499     try:
500         ensure_logged_in(page)
501         print(f"Loading {people_url}")
502         page.goto(people_url, wait_until="domcontentloaded")
503         company_name = get_company_name(page, fallback=slug)
504         company_id = get_company_id(page)
505         print(f"Company: {company_name} (slug={slug}, id={company_id or 'unknown'})")
506         matcher = CompanyMatcher(
507             names=[company_name, slug_to_name(slug), *args.alias],
508             company_id=company_id,
509             slug=slug,
510             threshold=args.match_threshold,
511             allow_prefix=args.allow_prefix_match,
512         )
513         print(f"Matching experience entries against: {matcher.describe()}")
514         known_urls = {u for (c, u) in existing if c == company_name}
515         print(f"{len(known_urls)} profiles for this company already saved.")
516         people = collect_people(page, people_url, args.max_people, known_urls)
517         print(f"{len(people)} new profiles to process.")
518         for i, (url, full_name) in enumerate(people.items(), 1):
519             first, last = split_name(full_name)
520             print(f"[{i}/{len(people)}] {full_name}")
521             job_title = None
522             if not args.skip_titles:
523                 job_title = get_job_title(page, url, matcher)
524                 print(f"    title: {job_title or '(not found)'}")
525                 polite_sleep(args.min_delay, args.max_delay)
526             email = None
527             if not args.skip_emails:
528                 email = get_email(page, url)
529                 if email:
530                     print(f"    email: {email}")
531                 polite_sleep(args.min_delay, args.max_delay)
532             row = {
533                 "company_name": company_name,
534                 "first_name": first,
535                 "last_name": last,
536                 "job_title": job_title or "",
537                 "linkedin_profile_url": url,
538                 "email": email or "",
539             }
540             append_csv(csv_path, row) # saved immediately
541             if db_path:
542                 write_sqlite(db_path, [{"row", "job_title": job_title, "email": email}])
543         finally:
544             ctx.close()
545         print(f"\nDone. Results are in {csv_path} + {f' and {db_path}' if db_path else ''}")
546 if __name__ == "__main__":
547     main()
548

```